Hardening of PIN-Derived Key Protection Against Offline Brute-Force Attacks: A Controlled Comparison of PBKDF2 and Argon2id

N. Janvitha Reddy
Roll No. AV.EN.U4CCE23027
Department of Electrical and Communication Engineering
Amrita Vishwa Vidyapeetham

P. Hima Varshini
Roll No. AV.EN.U4CCE23031
Department of Electrical and Communication Engineering
Amrita Vishwa Vidyapeetham

K. Sravya Reddy
Roll No. AV.EN.U4CCE23041
Department of Electrical and Communication Engineering
Amrita Vishwa Vidyapeetham

***Abstract*—Short numeric PINs are weak secrets, so the key derivation function (KDF) that turns a PIN into an encryption key becomes the main barrier against guessing. Kim and Choi [1] showed that on Windows devices without TPM protection, Windows Hello private-key files can be attacked offline, with PBKDF2 (10,000 iterations) as the key-derivation step. This project builds a controlled experimental model inspired by that protection layer: a vault using a PIN, a random salt, a KDF and AES-GCM, plus an automated offline cracker. We compare a PBKDF2-HMAC-SHA512 baseline with Argon2id at memory costs of 16, 32, 64 and 128 MiB. On a Windows 11 machine, a KDF-only benchmark of 10 trials per setting gave about 90.6 guesses/s for PBKDF2, while Argon2id fell from 50.6 to 7.7 guesses/s as memory increased, up to 11.8 times slower than the baseline. An end-to-end attack on the 4-digit PIN 1234 needed 1,235 guesses in every case, but took roughly 13–15 s with PBKDF2 and 134–142 s with Argon2id (64 MiB). Argon2id does not stop brute force; in our CPU-only setting it raises the cost of every guess.**

***Index Terms*—PIN, key derivation function, PBKDF2, Argon2id, offline brute-force attack, Windows Hello**

I. Introduction

Many systems let users unlock an account or a device with a short PIN instead of a long password, because a PIN is quick to type and easy to remember. The drawback is size. A 4-digit PIN has $10 \times 10 \times 10 \times 10 = 10{,}000$ possible values, and even a 6-digit PIN has only one million. Anyone who is able to test guesses quickly can eventually try every value.

Whether a fast guesser is realistic depends on where the guesses are checked. In an online attack the attacker must talk to the real system, which can limit attempts, add delays or lock the account. In an offline attack the attacker has already obtained the protected data, for example an encrypted key file copied from a disk, and checks guesses on their own computer. No server is involved, so there are no lockouts and no attempt counter. The only thing slowing the attacker down is how expensive it is to test one guess.

In a PIN-protected design, that cost is set by the KDF. The PIN is combined with a salt and passed through the KDF to produce a key, and the key protects the real secret. Our base paper, Kim and Choi [1], analyses Windows Hello on devices where hardware protection (a TPM) is unavailable and shows that the PIN-protected private-key files can be guessed offline. They claim that short PINs can still be recovered in ”realistic time” because of their key-derivation step, which they use PBKDF2 with 10,000 iterations.

This raised a practical question for us: if the guess is made more expensive by changing the KDF, how much does the attacker's guessing rate actually drop? We chose Argon2id, a memory-hard KDF whose memory cost can be set by the designer, and measured it against the PBKDF2 baseline in a controlled setting. We do not implement Windows Hello itself. Our vault is a simplified model of the cryptographic protection layer that [1] describes (Fig. 1).

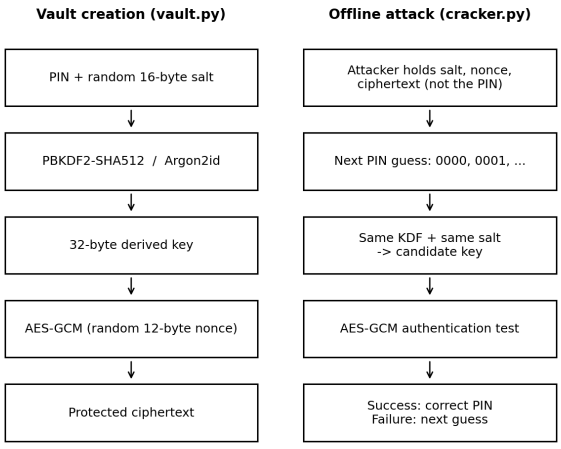


Fig. 1. Controlled experimental model used in this project.

A. Research Question and Hypothesis

Research question: How does replacing PBKDF2-based key derivation with a memory-hard KDF such as Argon2id affect the computational cost of offline PIN guessing in a controlled experimental model?

Hypothesis: Increasing Argon2id's memory cost increases the cost of each PIN guess and reduces the number of guesses an attacker can perform per second.

B. Contributions of This Project

- A controlled experimental model (vault, offline cracker and benchmark) of PIN-derived key protection, inspired by the base paper [1] and not a Windows Hello implementation.
- A PBKDF2-HMAC-SHA512 baseline (10,000 iterations) following the key-derivation step described in [1].
- A parameter study of Argon2id at 16, 32, 64 and 128 MiB, with 10 timed trials per setting.
- An end-to-end controlled attack on a 4-digit PIN for both KDFs, kept separate from the KDF-only benchmark.
- A discussion of what the results do and do not show, including limitations and extrapolated (not measured) estimates for longer PINs.

II. Literature Review

A. The Base Paper: Windows Hello on Hardware-Unsupported Devices

Kim and Choi [1] present a detailed analysis of Windows Hello, Microsoft's implementation of FIDO2-style passwordless login, on a device where TPM protection is unavailable. Their central finding is a "migration attack": with administrator privilege on the victim's machine, an attacker extracts the authentication data, recovers the PIN, and moves the data to another device to access the victim's Microsoft account.

The part relevant to us is the PIN-recovery step. The private keys are stored in files that hold, among other things, key properties (a salt and an iteration count) and the encrypted private key. As shown in Fig. 2, the key encryption key (KEK) is computed from the PIN or a seed through PBKDF2 and then SHA-512, and the KEK is used to unprotect the private key. A KEK is simply a key derived from the user's secret that is used to protect another key. In the paper's evaluation, PBKDF2 repeats the hash function 10,000 times, and their cracking tool needs about one second to decrypt an encrypted key file 100 times on a single processor core [1]. Because the guessing works on offline files, it can run "without any limits on the number of attempts" [1].

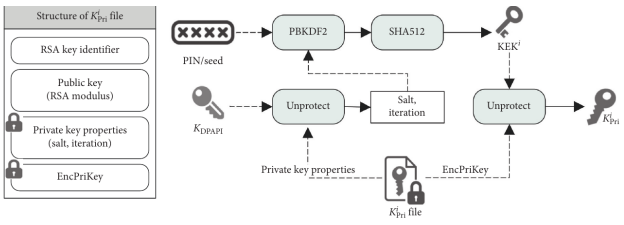


Fig. 2. Private-key decryption procedure in the base paper (Fig. 6(b) region of Kim and Choi [1]; reproduced for explanation with attribution).

The paper's contribution is the attack and its measurements. It does not set out to compare alternative KDFs, and a hardening comparison is outside its stated scope. Our work is therefore an experimental extension: we take the key-derivation layer the paper describes and measure what changes when the KDF is swapped. We do not claim the authors were unaware of memory-hard functions, only that evaluating one in this setting is not what the paper does.

TABLE I
Brute-Force Attack Duration by PIN Length, as Reported in the Base Paper [1] (Table 3 of the Paper)

# of digits	4	5	6	7	8
Time	1.7 min	17 min	2.8 hours	1.1 days	11 days

B. Password/PIN-Based Key Derivation and PBKDF2

A password-based KDF turns a low-entropy secret into a cryptographic key. NIST SP 800-132 [3] gives recommendations for deriving keys from passwords for storage applications, including the use of a random salt and an iteration count that makes each derivation deliberately slow. The pseudorandom function specified in PKCS #5 and RFC 8018 [2] is applied repeatedly to the password and salt by PBKDF2. The iteration count is the determinant of its cost and so, each guess in the game costs primarily CPU time.

The limitation is that PBKDF2 needs very little memory. Attackers who can run many guesses in parallel on specialised hardware gain a large advantage, because each guess needs only a small amount of state. Raising the iteration count helps but scales cost only in CPU time. We did not test GPUs, so this is background motivation from the literature and not a result of ours.

C. Gap Addressed by This Project

The literature gives us (a) a demonstrated offline PIN-guessing scenario around PBKDF2 [1], (b) specifications for PBKDF2 [2], [3], and (c) the design of Argon2id [4], [5]. What we could not find in the base paper is a measured comparison of a memory-hard KDF in this specific offline PIN-guessing setting. Our work fills that narrow gap with a controlled local experiment.

III. Methodology

A. Threat Model and Assumptions

- The attacker has obtained the protected data: salt, nonce and ciphertext. Nothing about these needs to be secret.
- The attacker knows the algorithms (KDF and AES-GCM) and their parameters.
- The only unknown is the PIN. The attacker does not guess the salt or nonce; the salt is an input the attacker already holds and reuses for every guess.
- There is no server, lockout or attempt limit during the attack (the offline setting).
- The attack is a CPU-only, single-process search on one laptop. We make no claims about GPUs or other special hardware.

B. System Overview

The system has three programs. `vault.py` creates a protected vault. `cracker.py` plays the offline attacker against that vault. `benchmark.py` measures how long each KDF takes and converts the result into guesses per second. Fig. 3 compares the two KDF workflows we evaluated, and Fig. 4 shows the cracker's loop.

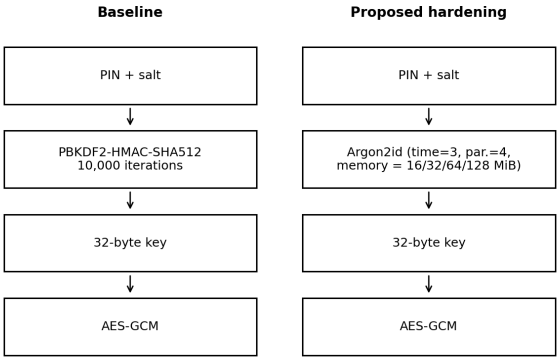


Fig. 3. PBKDF2 baseline and Argon2id workflows compared. Only the KDF block differs.

C. Vault Creation

To create a vault, the program generates a random salt, derives a 32-byte key from the PIN and salt with the chosen KDF, generates a random nonce, and encrypts a dummy secret ("This is our protected private key.") with AES-GCM. The vault record contains the salt, nonce, ciphertext and KDF name. The PIN and the derived key are not stored.

D. Offline Cracker

The cracker receives the salt, nonce and ciphertext. It generates every PIN of the required length as a zero-padded string in order (0000, 0001, . . . , 9999), derives a candidate key from each guess with the same KDF and salt, and tries to decrypt with AES-GCM. A failed authentication raises an exception and the loop continues. When decryption succeeds, the guess is the PIN. Since our test PIN is 1234 and enumeration starts at 0000, it is found on guess number 1,235.

E. Benchmark Design

The benchmark times the KDF alone. For each configuration it performs one warm-up call and then 10 timed calls using `time.perf_counter()`, with one random salt used for all measurements. It reports the mean, sample standard deviation, minimum and maximum, and defines the guessing rate as 1 divided by the mean time per derivation. The slowdown factor is the mean time of a configuration divided by the PBKDF2 mean time. Argon2id is tested at 16, 32, 64 and 128 MiB.

Equation (1) states the guessing-rate definition used throughout:

$$\text{guesses/s} = \frac{1}{\text{mean time of one KDF derivation (s)}} \qquad (1)$$

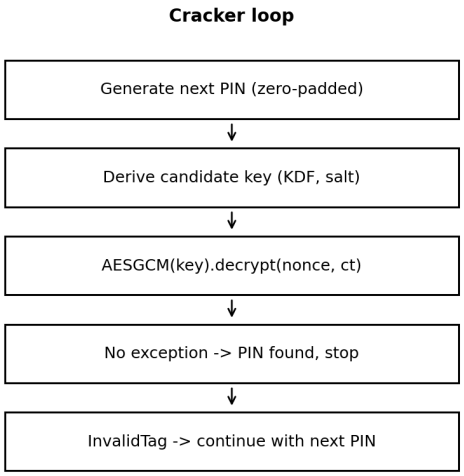


Fig. 4. Offline cracker workflow.

IV. Implementation Details

The code is written in Python. It uses `hashlib` for PBKDF2, the `argon2-cffi` package for Argon2id, and the `cryptography` package for AES-GCM. Only the relevant excerpts are shown below; the full files accompany this report.

TABLE II
Experimental Configuration

Parameter	Value
Test PIN / length	1234 / 4 digits (10,000 candidates)
Salt / nonce	16 bytes random / 12 bytes random
Derived key length	32 bytes
Baseline KDF	PBKDF2-HMAC-SHA512, 10,000 iterations
Argon2id parameters	`time_cost = 3`, `parallelism = 4`, memory 16/32/64/128 MiB
Authenticated encryption	AES-GCM
Benchmark trials	10 per configuration, plus 1 warm-up
Platform	Windows 11, Python 3.13.1
Hardware / library versions	Intel Core i5-10210U @ 1.60 GHz, 7.76 GB RAM, argon2-cffi 25.1.0, cryptography 50.0.2

A. PBKDF2 Derivation (`vault.py`)

```
def derive_key_pbkdf2(pin, salt):
    return hashlib.pbkdf2_hmac(
        "sha512",
        pin.encode(),
        salt,
        10_000,
        dklen=32
    )
```

This calls Python's built-in PBKDF2 with HMAC-SHA512, the PIN as the password, 10,000 iterations and a 32-byte output. The iteration count and hash match the base paper's description [1]. Our model does not reproduce Windows' full key-file format.

B. Argon2id Derivation (`vault.py`)

```
def derive_key_argon2id(pin, salt,
                  memory_kib=64 * 1024):
   return hash_secret_raw(
      secret=pin.encode(),
      salt=salt,
      time_cost=3,
      memory_cost=memory_kib,
      parallelism=4,
      hash_len=32,
      type=Type.ID
   )
```

The `memory_cost` argument is in KiB, so the benchmark passes `memory_mib` × `1024`. The default is 64 MiB. Time cost and parallelism are fixed so that memory is the one parameter we vary.

C. Vault Creation (`vault.py`)

```
def create_vault(pin, secret, kdf="pbkdf2"):
   salt = os.urandom(16)
   if kdf == "pbkdf2":
      key = derive_key_pbkdf2(pin, salt)
   elif kdf == "argon2id":
      key = derive_key_argon2id(pin, salt)
   nonce, ciphertext = encrypt_secret(secret, key)
   return {"salt": salt, "nonce": nonce,
         "ciphertext": ciphertext, "kdf": kdf}
```

Error handling for an unsupported KDF name is omitted from this excerpt. The function generates the salt, derives the key and encrypts the secret. Inside `encrypt_secret`, a fresh 12-byte nonce is produced with `os.urandom(12)` and `AESGCM(key).encrypt()` is used.

D. PIN Generation and Cracking Loop (`cracker.py`)

```
def generate_pins(length):
   for number in range(10 ** length):
      yield f"{number:0{length}d}"

# inside crack_pin(...):
for pin in generate_pins(pin_length):
   guesses += 1
   key = derive_key_pbkdf2(pin, salt)
   # or derive_key_argon2id
   aes = AESGCM(key)
   try:
      aes.decrypt(nonce, ciphertext, None)
      return {"pin": pin, "guesses": guesses, ...}
   except Exception:
      continue
```

`generate_pins` yields every zero-padded PIN in order. For each of them, the cracker formulates a candidate key and try to decode the text using AES-GCM. If the key is wrong, it throws an exception (authentication failure) and the loop continues; otherwise it returns the PIN and the numbers of guesses made and the time in the loop. The excerpt has been shortened: In real code, the user selects the desired KDF from a `kdf` argument.

E. Correctness Checks (`validation.py`)

Before using any timing results we ran `validation.py` to ensure: PBKDF2 matches the RFC 6070 test vector (using HMAC-SHA1 – as required by the vector, since it is the only primitive that requires it for the test vector to match); the same PIN and salt gives the same key for both KDFs; a different PIN results in a different key; both vault types decrypt correctly with the right PIN; a wrong PIN is rejected; and the cracker can find a known 2-digit PIN. All 8 checks passed (Fig. 5).

```
[PASS] RFC 6070 PBKDF2-HMAC-SHA1 vector
[PASS] PBKDF2: same PIN and salt produce same key
[PASS] Argon2id: same PIN and salt produce same key
[PASS] Different PIN produces a different key
[PASS] PBKDF2 vault decrypts with correct PIN
[PASS] Argon2id vault decrypts with correct PIN
[PASS] Wrong PIN is rejected by both vault types
[PASS] Cracker finds the known two-digit PIN

Validation result: 8/8 checks passed
```

Fig. 5. Terminal screenshot showing the validation results.

V. Results and Analysis

We report two kinds of measurement and keep them separate. The KDF-only benchmark (Section V-A) times one key derivation and is the basis for the guessing rates and graph. The end-to-end cracker runs (Section V-B) time a complete attack on one vault, including PIN generation, key derivation, AES-GCM and loop overhead. All values come from the final benchmark dataset (`kdf_benchmark_summary.csv`).

A. KDF-Only Benchmark

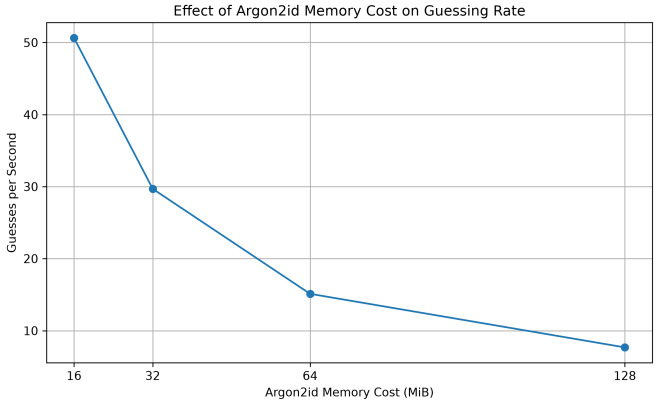


Fig. 6. Effect of Argon2id memory cost on measured guessing rate (project benchmark graph).

The guessing rate falls at every step: from 50.63 guesses/s at 16 MiB to 7.68 guesses/s at 128 MiB. Each doubling of memory roughly halves the rate, which is about what we would expect if the time is dominated by filling memory. Compared with PBKDF2, Argon2id at 16 MiB is already 1.79 times slower per guess, and at 128 MiB it is 11.79 times slower. This supports our hypothesis for this machine and these parameters.

The standard deviation shows how steady the measurements were. It is small compared with the mean for most settings (for example 2.60 ms against 33.69 ms at 32 MiB), but larger at 128 MiB (24.81 ms against 130.17 ms; maximum 198.61 ms), meaning one or more trials were noticeably slower than the rest. We ran on an ordinary Windows 11 machine with other processes active, so some timing noise is expected, and we did not control CPU frequency or background load. With only

TABLE III
KDF-ONLY BENCHMARK RESULTS (FINAL DATASET, 10 TRIALS PER SETTING; TIMES IN MILLISECONDS)

Configuration	Mean (ms)	Std (ms)	Min (ms)	Max (ms)	Guesses/s	Slowdown
PBKDF2 (10,000 it.)	11.04	1.16	10.20	13.75	90.55	1.00×
Argon2id, 16 MiB	19.75	4.00	15.64	25.24	50.63	1.79×
Argon2id, 32 MiB	33.69	2.60	29.91	37.53	29.69	3.05×
Argon2id, 64 MiB	66.15	6.25	58.31	76.40	15.12	5.99×
Argon2id, 128 MiB	130.17	24.81	114.68	198.61	7.68	11.79×

10 trials, we treat the means as indicative values and not as precise constants.

B. End-to-End Controlled Attack

We then ran `cracker.py` against a PBKDF2 vault and an Argon2id vault, both protecting the same dummy secret with PIN 1234. In both runs the cracker returned PIN 1234 after 1,235 guesses, which confirms the cracker is correct and that the search space is identical for both KDFs.

TABLE IV
CONTROLLED END-TO-END ATTACK VALIDATION (PIN 1234, 1,235 GUESSES)

KDF	Run	Time (s)	Guesses/s
PBKDF2	1	14.69	84.04
PBKDF2	2	13.19	93.65
Argon2id (64 MiB)	1	142.21	8.68
Argon2id (64 MiB)	2	134.26	9.20

PBKDF2 took about 13–15 s and Argon2id (64 MiB) about 134–142 s, roughly ten times longer for the same 1,235 guesses. The end-to-end PBKDF2 rate (84–94 guesses/s) is consistent with the benchmark rate of 90.55 guesses/s. The Argon2id end-to-end rate (8.7–9.2 guesses/s) is lower than the 15.12 guesses/s measured by the benchmark at 64 MiB. We have not isolated the reason. Possible contributors include system load during the long run, repeated memory allocation across consecutive derivations, or different run conditions, but these are untested explanations. We therefore do not treat the two numbers as the same measurement.

C. Estimates for Longer PINs (Extrapolation)

Running a full 8-digit attack would take days, so we did not run one. Instead, Table V divides the size of the search space by the benchmark guessing rate (time $= 10^n \div$ rate). These are worst-case estimates for trying every PIN on one core at the measured KDF-only rate, not measured attacks, and they ignore loop overhead (see the Argon2id gap in Section V-B) and any parallelism.

For comparison, the base paper reports 1.7 minutes for 4 digits, 2.8 hours for 6 digits and 11 days for 8 digits (Table I) using its own tool and hardware [1]. Our PBKDF2 estimates (110 s, 3.1 h and 12.8 days) are of the same order, which is a reasonable sanity check, although the machines differ. Under the same assumptions, Argon2id at 128 MiB raises the 8-digit estimate from about 12.8 days to about 150.7 days. This shows a cost increase, not prevention: the attacker still finishes the search.

TABLE V
ESTIMATED EXHAUSTIVE-SEARCH TIME (EXTRAPOLATED FROM TABLE III RATES; NOT MEASURED; DIG. = DIGITS, D = DAYS)

Configuration	4 dig.	6 dig.	8 dig.	8 dig. (d)
PBKDF2	110 s	3.1 h	306.8 h	12.8
Argon2id, 16 MiB	198 s	5.5 h	548.7 h	22.9
Argon2id, 32 MiB	337 s	9.4 h	935.7 h	39.0
Argon2id, 64 MiB	662 s	18.4 h	1,837.6 h	76.6
Argon2id, 128 MiB	1,302 s	36.2 h	3,615.8 h	150.7

VI. CONCLUSION

The base paper [1] demonstrated that, on Windows devices without TPM protection, PIN-protected private-key files can be attacked offline, with PBKDF2 as the key-derivation barrier. We built a controlled model of that protection layer with a vault, an offline cracker and a benchmark, used PBKDF2-HMAC-SHA512 with 10,000 iterations as the baseline, and added Argon2id with memory costs from 16 to 128 MiB.

In the final KDF-only benchmark, the guessing rate fell from 90.55 guesses/s for PBKDF2 to between 50.63 and 7.68 guesses/s for Argon2id, a slowdown of 1.79 to 11.79 times. End-to-end attacks on PIN 1234 needed 1,235 guesses with both KDFs, but took about 13–15 s with PBKDF2 and about 134–142 s with Argon2id at 64 MiB.

Argon2id did not eliminate brute-force guessing; rather, increasing its memory cost increased the computational and resource cost of each offline PIN guess and reduced the measured guessing rate in our controlled environment.

REFERENCES

[1] E. Kim and H.-K. Choi, "Security analysis and bypass user authentication bound to device of Windows Hello in the wild," *Security and Communication Networks*, vol. 2021, Art. no. 6245306, 13 pages, 2021. doi:10.1155/2021/6245306.

[2] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," RFC 8018, IETF, Jan. 2017.

[3] M. S. Turan, E. Barker, W. Burr, and L. Chen, "Recommendation for password-based key derivation, part 1: Storage applications," NIST Special Publication 800-132, Dec. 2010.

[4] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE Eur. Symp. Security and Privacy (EuroS&P)*, 2016.

[5] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 memory-hard function for password hashing and proof-of-work applications," RFC 9106, IETF, Sep. 2021.
[6] C. Percival, "Stronger key derivation via sequential memory-hard functions," presented at BSDCan, 2009.